

In-memory Blockchain: Toward Efficient and Trustworthy Data Provenance for HPC Systems

Abdullah Al-Mamun[†], Tonglin Li[‡], Mohammad Sadoghi[§], Dongfang Zhao^{†§}

[†]University of Nevada, Reno [‡]Lawrence Berkeley National Laboratory [§]University of California, Davis

Abstract—The state-of-the-art approaches for tracking data provenance on high-performance computing (HPC) systems are either supported by file systems or relational databases. These techniques shared the same critique on the provenance data's fidelity and the associated I/O overhead. This paper envisions to track the HPC data provenance using a distributed in-memory ledger—the core technique leveraged by blockchains and proven to be highly trustworthy by many large-scale applications. We pinpoint two system challenges—storage architecture and consensus protocol—for adopting blockchains to HPC and make the following contributions: (i) We design a new in-memory blockchain architecture for HPC systems, exploiting the high-performance network infrastructure InfiniBand and greatly reducing the I/O overhead; and (ii) We develop a new consensus protocol, namely proof-of-reproducibility (PoR), crafted for the new architecture, which takes into account both proof-of-work (PoW) and proof-of-stake (PoS) mechanisms. The correctness of PoR is both theoretically proven and experimentally verified. A prototype system is implemented and evaluated with more than one million transactions, showing 32× speedup compared to the filesystem-based provenance service and four orders of magnitude speedup compared to the database-based provenance service.

I. INTRODUCTION

The state-of-the-art approaches for tracking data provenance on high-performance computing (HPC) systems are either supported by filesystems [1, 2] or relational databases [3, 4]. These techniques shared the same critique on data fidelity: we do not know whether the provenance has been altered, either maliciously or unintentionally. Nevertheless, data fidelity is of prominent importance for many scientific applications deployed to HPC systems, as the data upon which scientific discovery rests must be trustworthy and retain its veracity at every point in the scientific workflow. To this end, developing an efficient and trustworthy data provenance service for HPC systems has been recently incentivized by various federal funding agencies, such as National Science Foundation [5] and U.S. Department of Energy [6].

One plausible approach to track scientific data provenance is through a distributed ledger—a core technique proven to be highly trustworthy by many blockchain-based applications without central governance. While blockchain itself exhibits many research and application opportunities (comprehensive reviews available from [7–9]), one of the impelling challenges for employing blockchains into HPC lies on the storage and I/O subsystem: existing blockchain frameworks replicate the entire chain of data on the local storage of all participating nodes. This design is hardly applicable to HPC systems because the (compute) node-local disks (a.k.a. burst buffers),

if any, are not for persisting large data but for buffering hot, usually small sized, data compared to those archived on the remote storage (e.g., Lustre [10], GPFS [11]). In other words, the above fact leads to two key challenges to adopt blockchains to HPC platforms:

- **Storage Architecture.** Most scientific applications are deployed to HPC clusters whose architecture is dramatically different than what existing blockchain systems are originally designed for, i.e., shared-nothing clusters. Many HPC systems comprise a set of tightly-coupled, and usually diskless, compute nodes interconnected with high-speed InfiniBand and protocols (RDMA), and all compute nodes can access a shared storage—usually a parallel file system in a remote cluster through a shared network. It is unrealistic, or prohibitively I/O-intensive (assuming the compute nodes simply take the mounted remote filesystems as local filesystems), for all compute nodes to fully replicate the blockchains on their local persistent disks.
- **Consensus Protocol.** Existing blockchain systems take various approaches to reach consensus such as proof-of-work (PoW), proof-of-stake (PoS), practical Byzantine fault-tolerance (PBFT), among others. It is unclear which, if any, of them would work appropriately for HPC systems where the majority of the applications are from domain sciences rather than for-profit sectors. For instance, the original PoW consensus (taken by Bitcoin [12]) asks the node to find a magical number (i.e., nonce) to satisfy a specific hashing function requirement (i.e., 17 leading zeros in the hashed value as of September 2018). The puzzle-solving procedure is designed to be compute-intensive, which is intentional for both security and business reasons but deemed a waste of time and resources for domain scientists.

This paper proposes a new blockchain architecture crafted for tracking data provenance on HPC systems. The new architecture is customized for the HPC environment: compute nodes have the option to maintain the blockchain without touching the local disks, such that both the persistent data size and the I/O overhead are greatly reduced. Furthermore, we design a new consensus protocol, namely, proof-of-reproducibility (PoR), for validating applications' data provenance. The key idea of PoR is that the consensus comes from not only the fellow compute nodes through PoW but also the remote shared storage through PoS. We prove the correctness of the

new consensus protocol, implement a system prototype with about 1,800 lines of Java code, and experimentally verify the system’s effectiveness. Results show that our proposed system outperforms the filesystem-backed provenance by up to $32\times$ and incurs four orders of magnitude smaller latency compared to the database-tracked provenance (Figure 3).

In the remainder of this paper, we review the related work in Section II. Section III gives a high-level overview of the proposed new architecture for in-memory blockchains. We present the new consensus protocols and prove their correctness in Section IV. Section V details the implementation of a prototype of the proposed new blockchain system. We report the experimental results in Section VI and finally conclude this paper in Section VII.

II. MOTIVATION AND RELATED WORK

Regardless of private (e.g., Hyperledger [13]) or public (e.g., Ethereum [14]), all existing blockchain systems assume the underlying computer infrastructure is shared-nothing: the memory subsystems and I/O subsystems are all independent on the participant nodes who are often connected through commodity networking such as Ethernet.

Recent blockchain research focuses on various system perspectives. In [15], a new framework is proposed to address the storage bloating problem in blockchains using Network Coded Distributed Storage (NC-DS). Another work [16] studies how quantum-computing-based attack to blockchains can be resisted. In [17], a fine-grained strategy is proposed to strengthen the reliability of blockchain architectures at the hardware level. In [18], a framework called Hawk is proposed to achieve transactional privacy from public blockchains. Ouroboros [19], a blockchain protocol based on proof-of-stake is proposed to provide stern security guarantees in blockchains and better efficiency than those blockchains based on proof-of-work. Inkchain [20] is designed as another permissioned blockchain solution that is inspired by Hyperledger [13] and enables customization and enhancement in arbitrary scenarios. To improve the reliability and achieve better fault tolerance, BigchainDB [21] is built upon the Practical Byzantine fault tolerant (PBFT) and comes with blockchain properties (e.g. decentralization, immutability, owner-controlled assets) as well as database properties (e.g. high transaction rate, low latency, indexing and querying of structured data). A 2-layer blockchain architecture (2LBC) is proposed in [22] to achieve stronger data integrity in distributed database systems based on a leader-rotation approach and proof-of-work. Unfortunately, none of the aforementioned work addresses the underlying platform architecture other than shared-nothing clusters assumed by existing blockchain systems. The proposed in-memory blockchain presented by this paper, therefore, for the first time, showcases a practical blockchain-like system crafted for data provenance on HPC systems.

In addition to our prior work on provenance [1, 2], we are well aware of recent advances in provenance management from the filesystem and database communities. In filesystems, Chen et al. [23, 24] proposed various I/O optimization for the

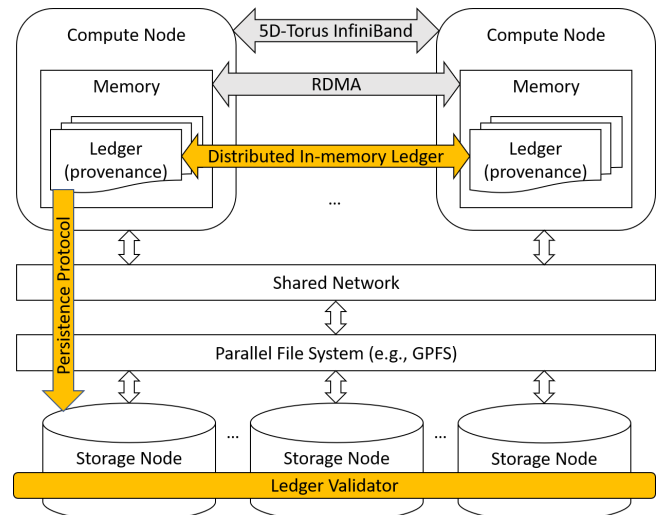


Figure 1. In-memory Blockchain Architecture.

storage-level provenance. In database, Glavic et al. [25, 26] proposed several techniques for improving the provenance in database transactions. In a more general sense, provenance is tightly related to areas like I/O subsystems and data management [27–32]. However, little literature exists to track provenance using a distributed ledger proposed by this work.

III. THE NEW BLOCKCHAIN ARCHITECTURE FOR HPC

Figure 1 illustrates the high-level overview of our envisioned distributed in-memory ledger, which is deployed to a typical HPC system. It should be clear that, although here we assume the compute nodes are diskless, some customized HPC systems do have node-local disks. For instance, a top-10 supercomputer called Cori [33] located at Lawrence Berkeley National Laboratory does have local SSD storage, namely, the burst buffer. Burst buffer is not a good candidate for ledgers because it is not designed for long-term data archival; it’s only purpose is to alleviate the I/O pressure for those I/O-intensive applications: cache the intermediate data locally. In addition, because scientific applications do not time-share the resources, the ledgers stored on the local storage (or, burst buffer) are usually purged after the job execution (for both performance and security reasons). In other words, even if a ledger can be technically persisted to local disk in some scenarios, that persistence is not permanent, which motivates us to come up with a secondary ledger and validator on the remote storage. Specifically, three key modules of our envisioned system are highlighted in the figure: a distributed in-memory ledger, a set of persistence protocols, and a remote persistent ledger. We will discuss each of them in more detail in the following.

In-memory Ledger. The first module is a distributed ledger implementation optimized for high-performance interconnect (e.g., InfiniBand) and protocols (RDMA) across compute nodes. In particular, the high-performance hardware should be able to significantly speed up communication-intensive consensus algorithms such as Practical Byzantine Fault Tolerance (PBFT) [34] widely used in permissioned blockchains (e.g.,

Hyperledger [13]). We are specifically interested in permissioned blockchains because the HPC environment for scientific applications allows only authenticated participants (as opposed to permissionless blockchain systems, e.g., Ethereum [14], which are open to the general public).

Persistence Protocols. The second module is the synchronization between the diskless compute nodes and the remote persistent storage. This synchronization has two purposes: (i) It increases the data reliability and (ii) It strengthens the data validity by the persistent ledger from the remote storage. The key question is how to persist the in-memory ledger into the remote parallel file system. The persistence cannot occur for every transaction as the enormous I/O's would become the performance bottleneck of the application.

Remote Validation. The third module focuses on the validation of the distributed ledger on the persistent storage. Because all compute nodes fetch and update the ledger only in volatile memory (or, in persistent local storage but with a short lifetime—purged after the job is complete), there has to be at least one ground truth on the (permanent) persistent storage (which cannot be compromised) to validate the ledger replicas in case of catastrophes (e.g., more than 50% of compute nodes crash and lose their ledgers). Note that the memory-only compute nodes are not necessarily less reliable than those with persistent storage; yet, the data stored on memory would get lost if the process who initiates the memory is killed.

IV. THE PoR CONSENSUS PROTOCOL FOR HPC SYSTEMS

As a co-design of the proposed new blockchain system architecture for scientific data management, we propose a new protocol for participant nodes to compete for validating scientific data provenance, namely, the winner (or, leader). Unlike existing blockchain systems who incentivize the competitors with financial reward, in scientific applications we need another way for such incentive. In essence, our proposed consensus protocol is a mix of PoW and PoS. In the context of blockchains, PoW refers to the fact that a specific node has spent a certain amount of time in working on a puzzle, while PoS indicates that a node has proven “shares” such as the time for which it has joined the network.

Our proposed consensus, called Proof-of-Reproducibility (PoR) consists of two phases of protocols. The first protocol, *Remote-Persist*, aims to leverage the shared storage to back up a ledger replica only in observation of all ledgers on compute nodes are essentially volatile. The second protocol, *Double-Validate*, further extends the first protocol by pushing the storage node into a more active position: whenever a new block is submitted, it will be validated by the storage node as well. Details of each protocol are as follows.

Remote Persist. The protocol is described in Protocol 1. In essence, a batch of new transactions are appended to the blockchain and persisted to a parallel file system (e.g., GPFS), as shown in Line 4 and Line 6. As discussed before, doing so adds one more layer of reliability to the data on volatile memory. However, while attaining high reliability, the system should not exhibit significant overhead. This is addressed in

Protocol 1 Persist in-memory blocks to shared storage

Require: Compute nodes M where the i -th node is M^i ; M_B^i the local blockchain on M^i ; a newly mined block b ; shared storage S ; S_B the blockchain copy on the storage;

Ensure: Validate b and persist it to S

```

1: function REMOTE-PERSIST( $b, M, S$ )
2:   for  $M_i \in M$  do
3:     if  $b$  is valid with  $M_B^i$  then
4:        $M_B^i \leftarrow M_B^i \cup b$   $\triangleright b$  is appended to the chain
5:       if  $b \notin S_B$  then
6:          $S_B \leftarrow S_B \cup b$ 
7:         Break  $\triangleright$  Storage is accessed only once
8:       end if
9:     end if
10:  end for
11: end function

```

Line 7, where we stop touching the shared storage as long as one copy of the new block is appended on the shared storage. Although the theoretical time complexity of this protocol is $O(|M|)$ and $|M|$ could be a fairly large number (e.g., tens of thousands of cores in leading-class supercomputers [33]), in practice it could finish in the first few iterations because of Line 7. We will demonstrate the effectiveness of the protocol in the evaluation section. The correctness of Protocol 1 is obvious because the only change to the original PoW consensus is the data persistence, which has nothing to do with the agreement between the compute nodes.

Double Validate. The PoR consensus is comprised of two pieces: the PoW from other compute nodes and the PoS from the shared storage (we can think of the shared storage as the oldest “node” in the network), as shown in Line 3 of Protocol 2. The benefits of this new consensus method are two folds: (i) A more reliable and stronger validity is achieved by the agreement from the persistent storage, which can be deemed as the ground truth of the permissioned blockchain; and (ii) It opens the road to reducing the network traffic across compute nodes because the endorsement from the shared storage can be considered as a heavy stake-holder and we could somewhat reduce the number of validations from other compute nodes. The time complexity of Protocol 2 is obviously $O(|M|)$, which is on par with the original PoW consensus. However, as mentioned above, i.e., benefit (ii), it is possible to reduce the number of iterations if we assign more weight to the shared storage.

Different than Protocol 1, the *Double-Validate* protocol does impact the agreement formation between participant nodes and in this case, including the storage node. Therefore, we need to demonstrate that the new protocol indeed leads to consensus, meaning that at least 50% of the compute nodes are guaranteed to hold the longest (same) ledgers. In the following, we will prove the correctness of the protocol, i.e., Theorem 1.

Theorem 1 (Correctness of PoR consensus). *If a n -node shared-nothing cluster is PoW-reliable to up to $\lfloor \frac{n}{2} \rfloor$ compro-*

Protocol 2 Validate blocks with remote storage

Require: Compute nodes M where the i -th node is M^i ; M_B^i the local blockchain on M^i ; a newly mined block b ; shared storage S ; S_B the blockchain copy on the storage;

Ensure: b is validated by both the local in-memory ledger M_B^i and the remote persistent ledger S_B , and then appended to all M_B 's and S_B .

```
1: function DOUBLE-VALIDATE( $b, M, S$ )
2:   for  $M_i \in M$  do
3:     if  $b$  is valid with  $M_B^i$  and  $S_B$  then
4:       if  $b \notin M_B^i$  then
5:          $M_B^i \leftarrow M_B^i \cup b$ 
6:       end if
7:       if  $b \notin S_B$  then
8:          $S_B \leftarrow S_B \cup b$ 
9:       end if
10:    end if
11:  end for
12: end function
```

mised nodes, then the extended $(n+1)$ -node cluster with one additional shared-storage node is also PoW-reliable to up to $\lfloor \frac{n+1}{2} \rfloor$ compromised nodes.

Proof. Without loss of generality, we assume the first half of the nodes (i.e., with indices $1, 2, \dots, \lfloor \frac{n}{2} \rfloor$) are compromised. We will show that the new storage node (index $n+1$), when implementing the proof-of-reproducibility protocol, will lead to at least $\lfloor \frac{n+1}{2} \rfloor$ nodes holding valid ledgers.

If n is indivisible by two, then it is a trivial problem to prove. Because $\lfloor \frac{n}{2} \rfloor \equiv \frac{n-1}{2}$, this implies that, according to the assumption, at least $\frac{n+1}{2}$, which is equal to $\lfloor \frac{n+1}{2} \rfloor$, compute nodes are valid. In other words, in the shared-storage cluster where there are totally $n+1$ nodes, we are guaranteed to have at least 50% of nodes holding the correct ledgers.

Now we consider the case where n is divisible by two, i.e., $\lfloor \frac{n}{2} \rfloor \equiv \frac{n}{2}$. We will prove this case by contradiction. Recall that we assume the first half of nodes are compromised; so, let us assume the storage node itself is compromised. That is, we assume nodes $1, 2, \dots, \frac{n}{2}, n+1$ are all compromised. According to Lines 3 and 7–10 in Protocol 2, the blocks in the ledger of the storage node all validated by all compute nodes, as opposed to the compute-node-only validation where more than 50% compute nodes could have validated a new block. As a consequence, if node $n+1$ is compromised, then nodes $\frac{n}{2} + 1, \frac{n}{2} + 2, \dots, n$ are not able to hold valid ledgers because all of them are cross-checked by node $n+1$. However, our assumption says the second half of the compute nodes are valid, which leads to a contradiction. Therefore, we prove that even if n is divisible by two, the shared-storage $(n+1)$ -node cluster can still tolerate up to $\lfloor \frac{n+1}{2} \rfloor$ compromised nodes. $\square$

V. SYSTEM IMPLEMENTATION

We have implemented a prototype system of the proposed blockchain architecture and consensus protocols with Java.

The source code, along with other details, can be found at the project website: <http://cse.unr.edu/hpdc/proj/imb>. At this point, we only release the very core modules of the prototype; some complimentary components and plug-ins are still being tested with more edge cases and will be released once they are stable enough. The current code base consists of more than 1,800 lines of code.

The prototype system currently runs on a virtualized environment where each node is executed with a user-level thread, local ledgers are stored on as (distinct) files, and network latency is throttled by a time delay parameterized by arbitrary statistical distributions (with average, variance, random seed). The prototype is being packaged into dockers such that it will be deployed to the production systems.

As proposed in [7], each blockchain system can be decomposed into four main components: ledger, consensus, cryptography, and smart contract. Because this paper focuses on ledger and consensus, we simply use SHA-256 [35] as the hashing function for the chained blocks. The current system implementation does not support smart contract: we wrap up the applications using pseudo transactions with dummy numerical values such that the prototype can take in a variety of applications. The remainder of this section focuses on ledger and consensus implementation.

The consensus protocol between compute nodes, as a building block of the proposed PoR consensus, shares the same essence of the conventional PoW but simplifies the compute-intensive puzzle taken by Bitcoin [12]. Specifically, our PoW does require a nonce number to mine and validate the new block. Our current PoW implementation is composed of the following three steps.

First, all the newly created transactions generated in a node added to that node as well as are propagated to other peers including the shared storage and eventually packed into a block by all the peers in the network for the preparation of starting the mining process.

Second, all the compute nodes including the shared storage will attempt to validate and add the block to its local replica of the blockchain. The node who will first be able to solve the puzzle (i.e., mining) for the newly created block, will append the block in its local in-memory blockchain and will receive the reward or credit. If the block does not get validated in this round, it is pushed in to a waiting queue in the respective compute node, so that the block can be processed by the node later. This is the step where we follow the conventional proof-of-work (PoW) among the compute nodes but in different way with a customized PoW (i.e., PoR) specially for the scientific data provenance, which is double-validated by the ground truth held by the storage node.

Finally, when the block is appended (i.e., after mining both by the node and the shared storage) successfully to the (local) blockchain by a node and the shared storage, it will then be propagated to the other peers in the network to make it validated; so that we can achieve the consensus. In essence, the first node who actually is able to validate the block will be the node appending it (i.e., the new block) to the local

blockchain and propagate the block to the entire network, and all other nodes in the network, after verifying the validity of the new block, will also update their local blockchains.

VI. EXPERIMENTAL RESULTS

A. Experiment Setup

Testbed. All experiments are carried out on Intel Core-i7 4.2 GHz CPUs along with 32 GB 2400 MHz DDR4 memory. The network latency of InfiniBand is two microseconds [36]; The network latency of Ethernet is 250 microseconds [37].

Evaluated Systems. We evaluate the proposed in-memory blockchain prototype against four other systems: two other blockchain systems and two provenance systems.

The first blockchain is a *Conventional Blockchain* system deployed to a shared-nothing cluster with Ethernet connections. The second blockchain is a *Hypothetical Blockchain* with high-performance networking interconnections (i.e., InfiniBand, RDMA) without any persistent storage; this is not a practical solution due to the lack of data persistence but is considered as the performance upper bound of the proposed *In-memory Blockchain*. We implement all three blockchain systems in Java and make reasonable effort in optimizing all of them.

The two provenance systems are built-in modules from database systems and file systems, respectively. The database provenance is called SPADE [4], tracking and querying provenance using a graph database. The filesystem provenance is called FusionProv [1], a provenance module built upon the FusionFS [38] distributed file system.

Workload. The benchmarks is in the transaction format, which is similar to funds transfer between bank accounts. At the beginning of the transaction, the system checks whether the submitted transaction is valid. If so, the statuses (balances) of two nodes are updated accordingly, followed by the propagation of the updates to all other nodes in the network. On average, each block contains about four transactions; each block, however, can have an arbitrary number of transactions between one and ten in our experiments.

B. Latency

We compare the proposed distributed in-memory ledger to two baseline systems at a small scale of seven compute nodes and one storage node. The metric we use to compare these systems is *responsiveness*, indicating the time for appending a new block to the chain. Note that responsiveness is negatively correlated to the popular blockchain metric *transaction throughput (txn/sec)*—the number of transactions that can be completed per second (Bitcoin is about seven txn/sec). We do not use txn/sec because it is application-dependent and highly impacted by the PoW difficulty (Bitcoin requires 17 leading zeros after being hashed with SHA-256 [35]).

We report the overall response time of all systems in Figure 2. The response time ranges (i.e., from minimum to maximum) of three systems are plotted on each of the seven nodes and the pseudo-node representing the shared storage. Note that the shared storage is inapplicable to shared-nothing

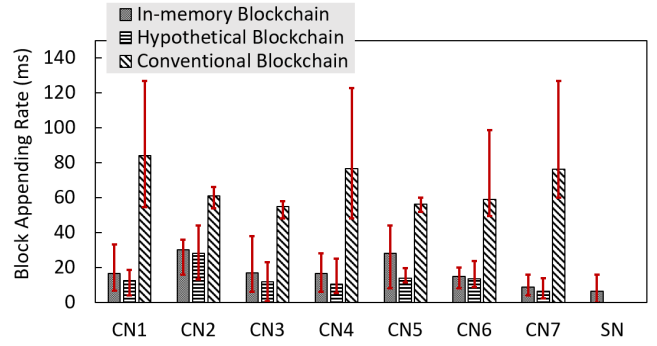


Figure 2. **Block appending rates at small scales: seven Compute Nodes (CNs) and one Storage Node (SN).** Histograms: average; Red bars: minimal and maximal values.

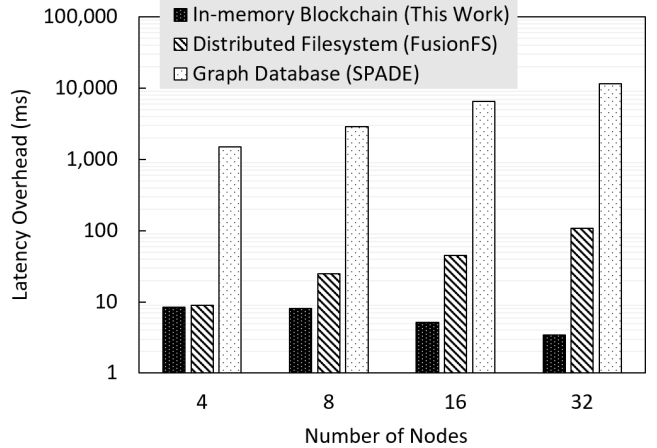


Figure 3. **Latency Overhead of In-memory Blockchain, Distributed Filesystem, and Graph Database.**

clusters; so, no numbers are reported for both systems in the figure. We can see that on all seven nodes in the conventional blockchain exhibit the slowest response time, with a big gap to the other two systems. In contrast, the hypothetical blockchain only takes less than 40 ms to append a block. Our proposed in-memory blockchain can append a block slightly slower than the upper bound from the ideal system, yet significantly faster than the conventional blockchains. For example, on Compute Node 7 the in-memory blockchain completes a new block appending in 8.9 ms while the shared-nothing blockchain completes that in 76.3 ms, leading to a 8.6 $\times$ speedup. One observation worth noting is that the response time of the shared storage is almost negligible (6.5 ms), which can be explained by that the storage node does not propagate the block to other peers.

C. Comparison to Filesystems and Databases

We report the latency overhead incurred by three types of provenance tools: (i) an in-memory blockchain proposed by this paper; (ii) a distributed filesystem for HPC [38]; and (iii) a relational database for graph processing [4]. The latency incurred by in-memory blockchain is only applicable to a single node, i.e., the winner who eventually appends the block, which is different than the other two systems where the latency

occurs on all nodes. To make it a fair comparison, we amortize the in-memory blockchain latency from the single node to all nodes. We only compare three systems on up to 32 nodes because the database-based provenance is not scalable beyond tens of nodes.

As we can see in Figure 3, the In-memory Blockchain incurs comparable latency overhead to Distributed Filesystem at small scales (four nodes), but then outperform the latter on eight and more nodes. In-memory Blockchain outperform Graph Database by orders of magnitude at all scales. At 32-node scale, the proposed in-memory blockchain is $32\times$ faster than the filesystem and four orders of magnitude than the database. Another desirable property of in-memory blockchain is the amortized per-node latency decreases with more nodes.

VII. CONCLUSION

This paper presents a new in-memory blockchain system architecture where ledgers are maintained mostly in memory and a new consensus protocol crafted for the new architecture. The new architecture and new consensus collectively enable an efficient blockchain-like ledger service for trustworthy data provenance on HPC systems. The correctness of the proposed consensus is both theoretically proven and experimentally verified. A lightweight system prototype is implemented and evaluated with more than one million transactions, showing $32\times$ speedup compared to the filesystem-based provenance service and four orders of magnitude speedup compared to the database-based provenance service.

Acknowledgement. This work is in part supported by a Microsoft Azure Research Award.

REFERENCES

- [1] D. Zhao, C. Shou, T. Malik, and I. Raicu, "Distributed data provenance for large-scale data-intensive computing," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2013.
- [2] C. Shou, D. Zhao, T. Malik, and I. Raicu, "Towards a provenance-aware distributed filesystem," in *TaPP Workshop, USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013.
- [3] S. Lee, S. Kohler, B. Ludascher, and B. Glavic, "A sql-middleware unifying why and why-not provenance for first-order queries," in *IEEE International Conference on Data Engineering (ICDE)*, 2017.
- [4] A. Gehani and D. Tariq, "SPADE: Support for Provenance Auditing in Distributed Environments," in *Proceedings of the 13th International Middleware Conference (Middleware)*, 2012.
- [5] NSF CICI, "<https://www.nsf.gov/pubs/2018/nsf18547/nsf18547.htm>," Accessed 2018.
- [6] DOE SBIR, "<https://www.sbir.gov/sbirsearch/detail/1307745>," Accessed 2017.
- [7] K. Zhang and H. Jacobsen, "Towards dependable, scalable, and pervasive distributed ledgers with blockchains," in *38th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2018.
- [8] S. Gupta and M. Sadoghi, *Blockchain Transaction Processing*. Springer International Publishing, 2018.
- [9] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "Blockbench: A framework for analyzing private blockchains," in *ACM International Conference on Management of Data (SIGMOD)*, 2017.
- [10] P. Schwan, "Lustre: Building a file system for 1,000-node clusters," in *Proceedings of the linux symposium*, 2003.
- [11] F. Schmuck and R. Haskin, "GPFS: A shared-disk file system for large computing clusters," in *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*, 2002.
- [12] Bitcoin, "<https://bitcoin.org/bitcoin.pdf>," Accessed 2018.
- [13] Hyperledger, "<https://www.hyperledger.org/>," Accessed 2018.
- [14] Ethereum, "<https://www.ethereum.org/>," Accessed 2018.
- [15] M. Dai, S. Zhang, H. Wang, and S. Jin, "A low storage room requirement framework for distributed ledger in blockchain," *IEEE Access*, vol. 6, pp. 22 970–22 975, 2018.
- [16] Y. Gao, X. Chen, Y. Chen, Y. Sun, X. Niu, and Y. Yang, "A secure cryptocurrency scheme based on post-quantum blockchain," *IEEE Access*, vol. 6, pp. 27 205–27 213, 2018.
- [17] J. Xiao, J. Lou, J. Jiang, X. Li, X. Yang, and Y. Huang, "Blockchain architecture reliability-based measurement for circuit unit importance," *IEEE Access*, vol. 6, pp. 15 326–15 334, 2018.
- [18] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou, "Hawk: The blockchain model of cryptography and privacy-preserving smart contracts," in *2016 IEEE Symposium on Security and Privacy (SP)*, 2016.
- [19] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Advances in Cryptology (CRYPTO)*, 2017.
- [20] Inkchain, "<https://github.com/inklabsfoundation/inkchain>," Accessed 2018.
- [21] BigchainDB, "<https://github.com/bigchaindb/bigchaindb>," Accessed 2018.
- [22] L. Aniello, R. Baldoni, E. Gaetani, F. Lombardi, A. Margheri, and V. Sassone, "A prototype evaluation of a tamper-resistant high performance blockchain-based transaction log for a distributed database," in *13th European Dependable Computing Conference (EDCC)*, 2017.
- [23] D. Dai, Y. Chen, P. Carns, J. Jenkins, and R. Ross, "Lightweight provenance service for high-performance computing," in *International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2017.
- [24] D. Dai, Y. Chen, D. Kimpe, and R. Ross, "Provenance-based object storage prediction scheme for scientific big data applications," in *IEEE International Conference on Big Data (BigData)*, 2014.
- [25] B. S. Arab, D. Gawlick, V. Krishnaswamy, V. Radhakrishnan, and B. Glavic, "Using reenactment to retroactively capture provenance for transactions," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 30, no. 3, pp. 599–612, March 2018.
- [26] X. Niu, R. Kapoor, B. Glavic, D. Gawlick, Z. H. Liu, V. Krishnaswamy, and V. Radhakrishnan, "Provenance-aware query optimization," in *IEEE 33rd International Conference on Data Engineering (ICDE)*, 2017.
- [27] P. Mehta, S. Dorkenwald, D. Zhao, T. Kaftan, A. Cheung, M. Balazinska, A. Rokem, A. Connolly, J. Vanderplas, and Y. AlSayyad, "Comparative evaluation of big-data systems on scientific image analytics workloads," in *Proceedings of the 43rd International Conference on Very Large Data Bases (VLDB)*, 2017.
- [28] T. Li, C. Ma, J. Li, X. Zhou, K. Wang, D. Zhao, and I. Raicu, "Graph/z: A key-value store based scalable graph processing system," in *Cluster Computing, IEEE International Conference on*, 2015.
- [29] Z. Zhou, X. Yang, D. Zhao, P. Rich, W. Tang, J. Wang, and Z. Lan, "I/O-aware batch scheduling for petascale computing systems," in *Cluster Computing, IEEE International Conference on*, 2015.
- [30] D. Zhao, J. Yin, K. Qiao, and I. Raicu, "Virtual chunks: On supporting random accesses to scientific data in compressible storage systems," in *IEEE International Conference on Big Data*, 2014, pp. 231–240.
- [31] D. Zhao, J. Yin, and I. Raicu, "Improving the i/o throughput for data-intensive scientific applications with efficient compression mechanisms," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC '13), poster session*, 2013.
- [32] T. Li, K. Keahey, K. Wang, D. Zhao, and I. Raicu, "A dynamically scalable cloud data infrastructure for sensor networks," in *Proceedings of the 6th Workshop on Scientific Cloud Computing (ScienceCloud)*, 2015.
- [33] The Cori Supercomputer, "<http://www.nersc.gov/users/computational-systems/cori>," Accessed 2018.
- [34] J. Sousa, A. Bessani, and M. Vukolic, "A byzantine fault-tolerant ordering service for the hyperledger fabric blockchain platform," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2018.
- [35] SHA-256, "<https://en.bitcoin.it/wiki/SHA-256>," Accessed 2018.
- [36] InfiniBand Latency, "http://www.mellanox.com/related-docs/whitepapers/WP_RoCE_vs_iWARP.pdf," Accessed 2018.
- [37] Ethernet Latency, "<http://www.evanjones.ca/ethernet-latency.html>," Accessed 2018.
- [38] D. Zhao, N. Liu, D. Kimpe, R. Ross, X.-H. Sun, and I. Raicu, "Towards exploring data-intensive scientific applications at extreme scales through systems and simulations," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 27, no. 6, 2016.